

Why?

grep?





Spark officially sets a new record in large-scale sorting

November 5, 2014 | by Reynold Xin



A month ago, we shared with you our entry to the 2014 Gray Sort competition, a 3rd-party benchmark measuring how fast a system can sort 100 TB of data (1 trillion records). Today, we are happy to announce that our entry has been reviewed by the benchmark committee and we have officially won the [Daytona GraySort contest!](#)

In case you missed our [earlier blog post](#), using Spark on 206 EC2 machines, we sorted 100 TB of data on disk in 23 minutes. In comparison, the previous world record set by Hadoop MapReduce used 2100 machines and took 72 minutes. This means that Spark sorted the same data **3X faster** using **10X fewer machines**. All the sorting took place on disk (HDFS), without using Spark's in-memory cache.

This entry tied with a UCSD research team building high performance systems and we jointly set a new world record.

	Hadoop MR Record	Spark Record	Spark 1 PB
Data Size	102.5 TB	100 TB	1000 TB
Elapsed Time	72 mins	23 mins	234 mins
# Nodes	2100	206	190
# Cores	50400 physical	6592 virtualized	6080 virtualized
Cluster disk throughput	3150 GB/s (est.)	618 GB/s	570 GB/s
Sort Benchmark Daytona Rules	Yes	Yes	No
Network	dedicated data center, 10Gbps	virtualized (EC2) 10Gbps network	virtualized (EC2) 10Gbps network
Sort rate	1.42 TB/min	4.27 TB/min	4.27 TB/min
Sort rate/node	0.67 GB/min	20.7 GB/min	22.5 GB/min



Big Data



Big Data

Big Code

```
public class WordCount {
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {
            String line = value.toString();
            StringTokenizer tokenizer = new StringTokenizer(line);
            while (tokenizer.hasMoreTokens()) {
                word.set(tokenizer.nextToken());
                context.write(word, one);
            }
        }
    }

    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {
            int sum = 0;
            for (IntWritable val : values) {
                sum += val.get();
            }
            context.write(key, new IntWritable(sum));
        }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "wordcount");

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        job.setMapperClass(Map.class);
        job.setReducerClass(Reduce.class);

        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
    }
}
```



Big Data

Big Code

```
public class WordCount {
    public static class Map extends Mapper<LongWritable, Text, Text, IntWritable> {
        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(LongWritable key, Text value, Context context) throws IOException,
            InterruptedException {
                String line = value.toString();
                StringTokenizer tokenizer = new StringTokenizer(line);
                while (tokenizer.hasMoreTokens()) {
                    word.set(tokenizer.nextToken());
                    context.write(word, one);
                }
            }
    }

    public static class Reduce extends Reducer<Text, IntWritable, Text, IntWritable> {
        public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {
                int sum = 0;
                for (IntWritable val : values) {
                    sum += val.get();
                }
                context.write(key, new IntWritable(sum));
            }
    }

    public static void main(String[] args) throws Exception {
        Configuration conf = new Configuration();

        Job job = new Job(conf, "wordcount");
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        job.setMapperClass(Map.class);
        job.setReducerClass(Reduce.class);

        job.setInputFormatClass(TextInputFormat.class);
        job.setOutputFormatClass(TextOutputFormat.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        job.waitForCompletion(true);
    }
}
```

Tiny Code

```
object WordCount{
    def main(def main(args: Array[String])){
        val sparkConf = new SparkConf()
            .setAppName("wordcount")

        val sc = new SparkContext(sparkConf)

        sc.textFile(args(0))
            .flatMap(_.split(" "))
            .countByValue
            .saveAsTextFile(args(1))
    }
}
```





Why Spark?

Readability

Expressiveness

Fast

Testability

Interactive

Fault Tolerant

Unify Big Data



Course Overview

- **Basics of Spark**
- **Core API**
- **Cluster Managers**
- **Spark Maintenance**
- **Libraries**
 - SQL
 - Streaming
 - MLlib/GraphX
- **Troubleshooting / Optimization**
- **Future of Spark**



Section Overview

- Basics of Spark
- Hadoop
- Core API
- History of Spark
- Cluster Managers
 - Installation
- Spark Maintenance
 - Big Data's Hello World
 - Course Prep
- Libraries
 - SQL
 - Streaming
 - MLlib/GraphX
- Troubleshooting / Optimization
- Future of Spark





The MapReduce Explosion

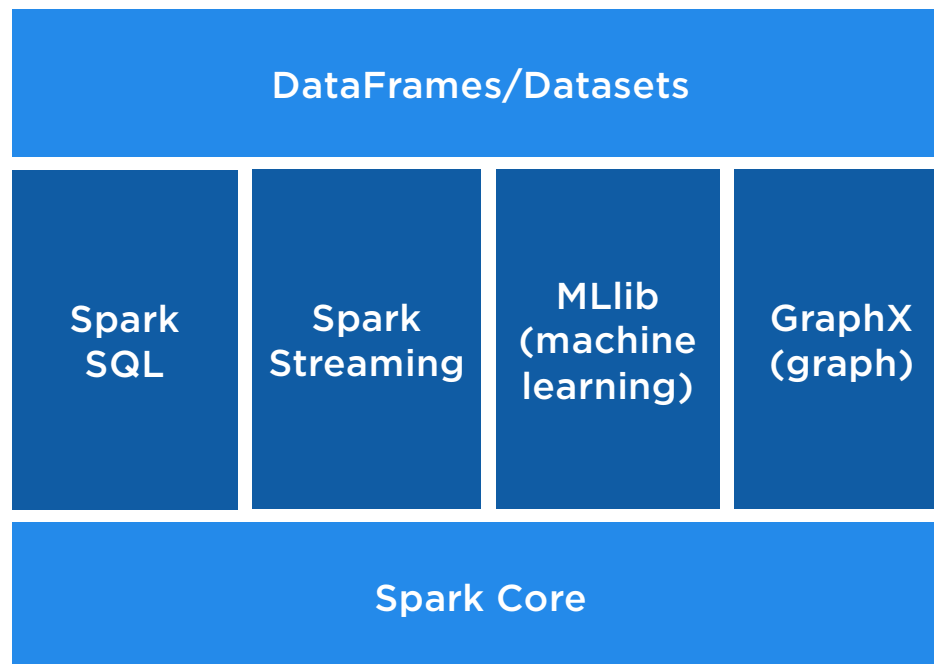




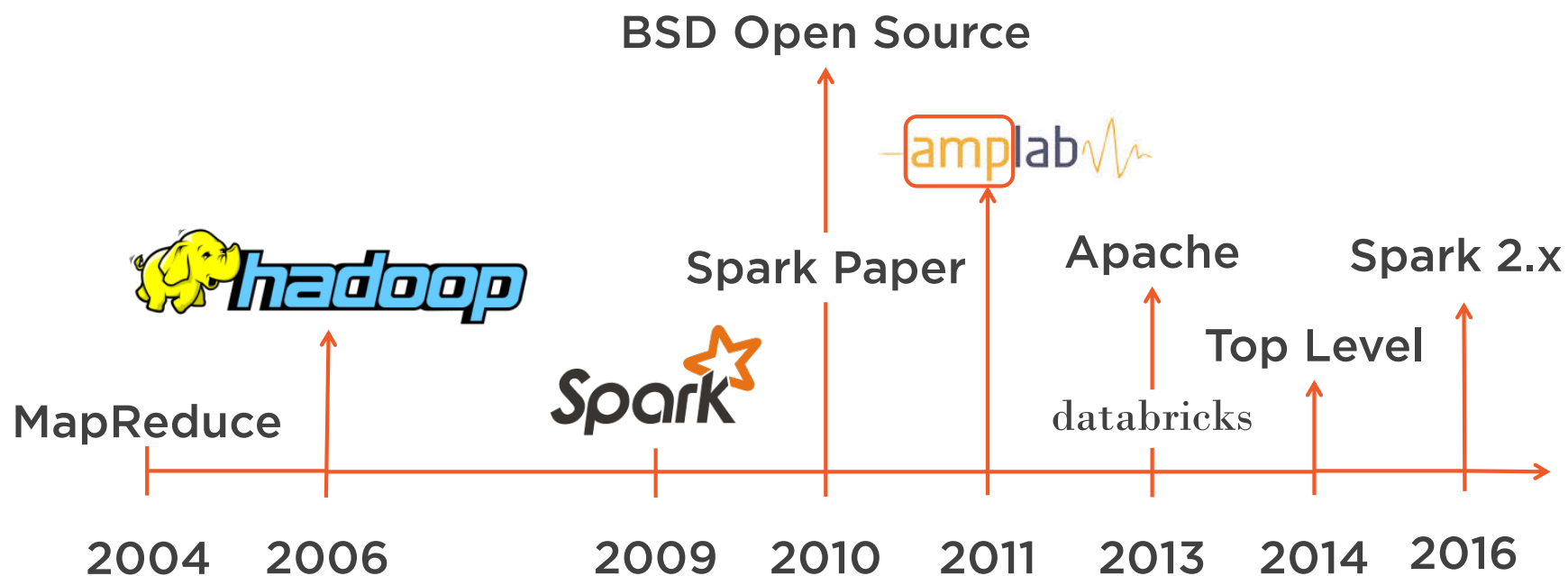




A Unified Platform for Big Data



The History of Spark



databricks == STABILITY



Stability

API Stability

Apache Spark 2.0.0 is the first release in the 2.X major line. Spark is guaranteeing stability of its non-experimental APIs for all 2.X releases. Although the APIs have stayed largely similar to 1.X, Spark 2.0.0 does have API breaking changes. They are documented in the [Removals, Behavior Changes and Deprecations](#) section.

Changes of behavior

- **Spark Core**
 - [\[SPARK-25088\]](#) Rest Server default & doc updates
- **Spark SQL**
 - [\[SPARK-23549\]](#) Cast to timestamp when comparing timestamp with date
 - [\[SPARK-24324\]](#) Pandas Grouped Map UDF should assign result columns by name
 - [\[SPARK-23425\]](#) load data for hdfs file path with wildcard usage is not working properly
 - [\[SPARK-23173\]](#) from_json can produce nulls for fields which are marked as non-nullable
 - [\[SPARK-24966\]](#) Implement precedence rules for set operations
 - [\[SPARK-25708\]](#) HAVING without GROUP BY should be global aggregate
 - [\[SPARK-24341\]](#) Correctly handle multi-value IN subquery
 - [\[SPARK-19724\]](#) Create a managed table with an existed default location should throw an exception

Please read the [Migration Guide](#) for all the behavior changes

Known Issues

- [\[SPARK-25271\]](#) CTAS with Hive parquet tables should leverage native parquet source

<https://spark.apache.org/releases/spark-release-MAJOR-MINOR-REVISION.html>



Stability



SparkQA commented 8 minutes ago

Test build #35017 has finished for PR 6841 at commit [276389d](#).

- This patch **fails MiMa tests**.
- This patch merges cleanly.
- This patch adds the following public classes (*experimental*):
 - `class JavaSampleActorReceiver<T> extends UntypedActor`
 - `public class JavaActorWordCount`
 - `class JavaGlobalActorSystem`
 - `trait ActorHelper extends Logging`
 - `class JavaActorHelper(actor: Actor)`
 - `case class Statistics(numberOfMsgs: Int,`
 - `class Supervisor extends Actor`
 - `trait ActorSystemFactory extends Serializable`

<https://github.com/apache/spark/pull/6841>



Stability



Who Is Using Spark?



Yahoo!

CONVIVA

NETFLIX

Goldman
Sachs

PANDORA

comcast

ebay

hhmi | janelia
Research Campus



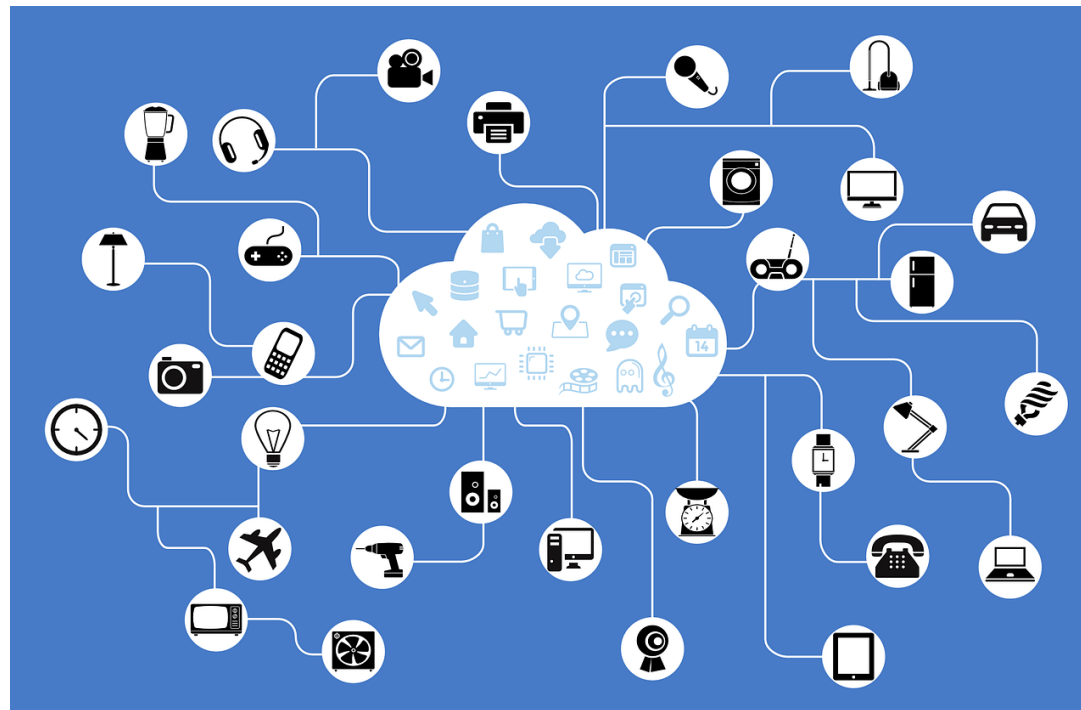
Spark Languages



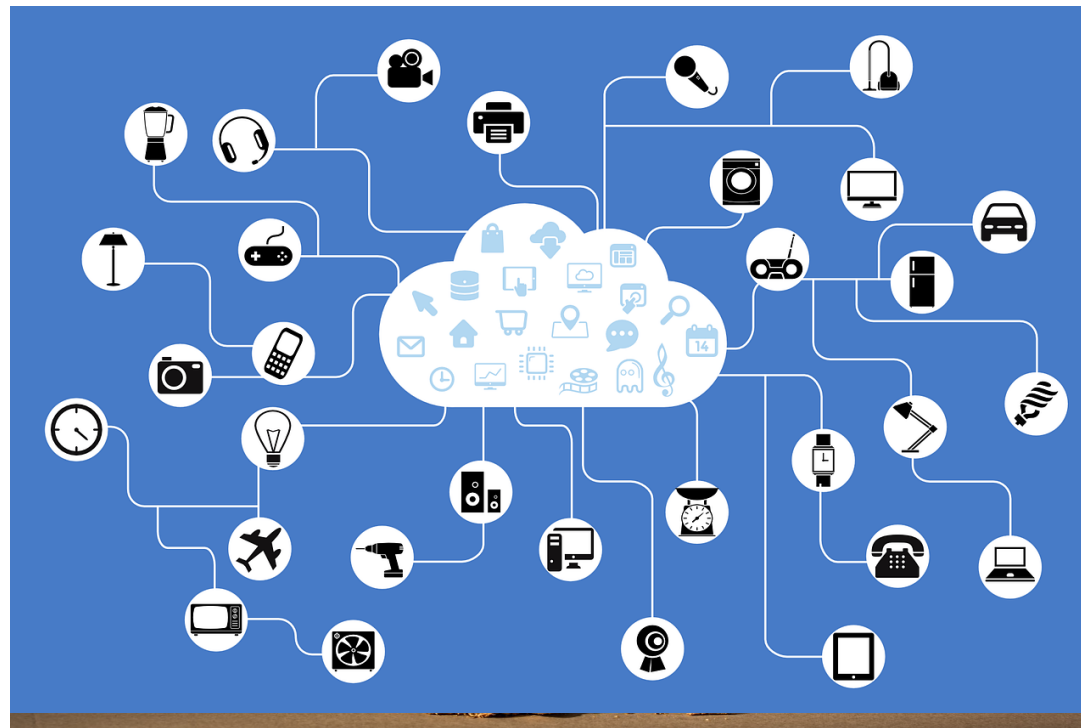
Spark Languages



Big Data



Big Data



Big Data



Course Notes



Spark Logistics



Resources

- <https://amplab.cs.berkeley.edu/for-big-data-moores-law-means-better-decisions/>
- https://www.chrisstucchio.com/blog/2013/hadoop_hatred.html
- <http://aadrake.com/command-line-tools-can-be-235x-faster-than-your-hadoop-cluster.html>
- <https://spark.apache.org>
 - </documentation.html>
 - </docs/latest/>
 - </community.html>
 - </examples.html>
- **Learning Spark: Lightning-Fast Big Data Analysis** by Holden Karau, Andy Konwinski, Patrick Wendell, Matei Zaharia
- <https://github.com/apache/spark>



Summary

- Why
- MapReduce Explosion
- Spark's History
- Installation
- Hello Big Data!
- Additional Resources

